Security Operation Centre (Soc) Using Siem and Soar

Project Guide -Vipul Kumar

Project No – 07

Name – Kuldeep Singh

What is Security Operation Centre (Soc) ?

- A Security Operations Center is the frontline defense of modern organizations—a centralized team operating around the clock to monitor networks, detect threats, and respond to security incidents before they cause damage.
- A Security Operations Center is the frontline defense of modern organizations—a centralized team operating around the clock to monitor networks, detect threats, and respond to security incidents before they cause damage.

Security Information and Event Management (SIEM)

SIEM platforms serve as the intelligence hub of security operations, aggregating aggregating and correlating vast amounts of log data from across your infrastructure to identify patterns that indicate potential threats.

- Log Collection
  - Aggregate security events from firewalls, servers, applications, and endpoints
- Event Correlation
  - Connect related events across systems to identify complex attack patterns
- Real-time Alerting
  - Generate immediate notifications when suspicious activity is detected
- Incident Visibility
  - Provide comprehensive view of security posture across the organization

Security Orchestration, Automation, Automation, and Response (SOAR)

SOAR transforms security operations by automating repetitive tasks and orchestrating complex response workflows. response workflows. This technology dramatically reduces the time between threat detection and containment. containment.

Byeliminating manual intervention for common scenarios, SOAR enables SOC teams to focus on sophisticated sophisticated threats requiring human analysis and decision-making.

- ❖ Alert Triggered
- ➢ Security event detected by monitoring systems
- ❖ Automated Analysis
- ➢ SOAR evaluates severity and context automatically
- ❖ Orchestrated Response
- ➢ Predetermined actions executed across security tools
- ❖ Rapid Containment
- ➢ Threat neutralized in seconds, not hours

Wazuh Fresh Installation – Successful Setup

Command use to install Wazuh :

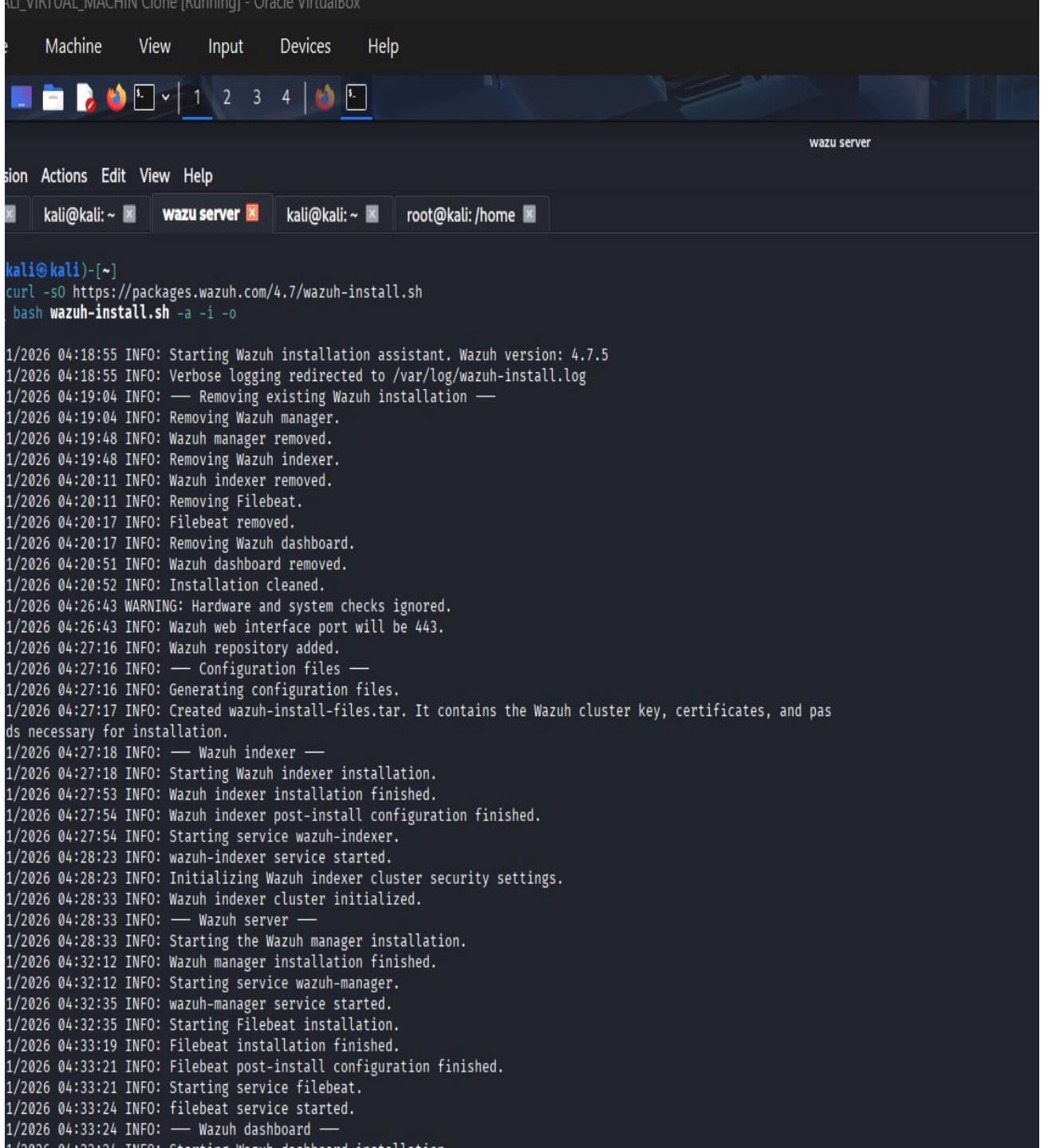
- curl -sO https://packages.wazuh.com/4.7/wazuh-install.sh
- sudo bash wazuh-install.sh -a -i -o

What This Script Does

- Performs a **clean installation** of Wazuh **version 4.7.5**
- Automatically installs **all core components**
  - Wazuh Manager
  - Wazuh Indexer (OpenSearch)
  - Filebeat
  - Wazuh Dashboard

Key Installation Steps (From Logs)

- Removed old Wazuh components (manager, indexer, dashboard)
- Cleaned previous configurations
- Generated new certificates and cluster keys
- Installed and started services in correct order:
  - Wazuh Indexer → Started
  - Wazuh Manager → Started
  - Filebeat → Started
  - Wazuh Dashboard → Installed



Editing Local Rules s Restarting Wazuh Manager

- **Command for Local rules**
- ➢ sudo vi /var/ossec/etc/rules/local_rules.xml
- **Explanation:**
- Opens **local_rules.xml** file in **vi editor**
- This file is used to:
  - ➢ Create **custom Wazuh rules**
  - ➢ Modify alert conditions
  - ➢ Detect specific attacks or events
- Changes here directly affect **log analysis and alert generation**
- **Command to restart Wazuh Manager service**
- ➢ sudo systemctl restart wazuh-manager
- **Explanation:**
- Restarts the **Wazuh Manager service**
- Required after editing rule files
- Reloads:
  - ➢ Custom rules
  - ➢ Decoders
  - ➢ Analysis engine configuration

```
┌──(kali㉿kali)-[~]
└─$ sudo vi  /var/ossec/etc/rules/local_rules.xml

sudo] password for kali:

┌──(kali㉿kali)-[~]
└─$ sudo systemctl restart wazuh-manager
```

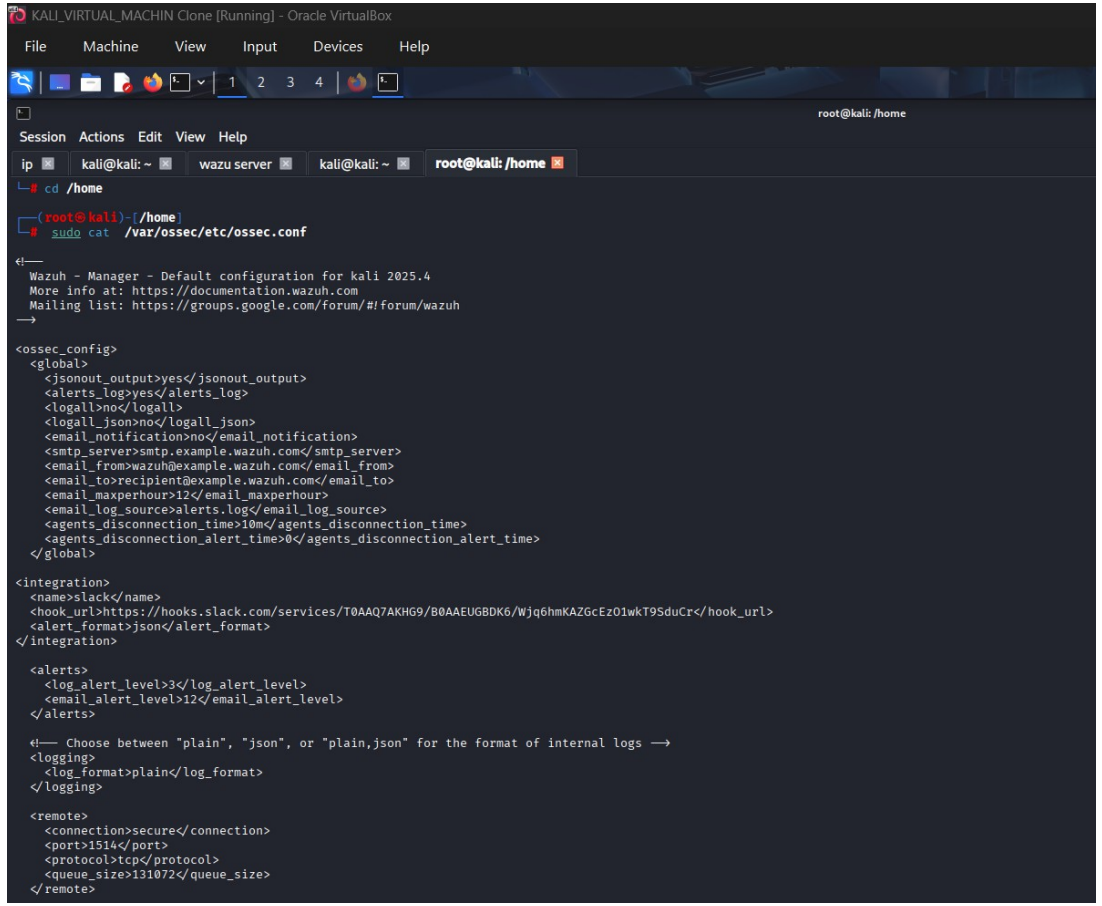
Installing Auditd (Linux Auditing Tool)

```
10  sudo apt -get install -y auditd
11  sudo apt-get install -y auditd

-(kali㉿kali)-[~]
$
```

- **Command use to install Auditd**
- ➢ sudo apt-get install -y auditd
- **What is auditd?**
- ❖ **auditd** is a Linux **system auditing service**
- ❖ It monitors and records:
  - ➢ File access
  - ➢ Command execution
  - ➢ User activities
  - ➢ Security-critical system changes
- **Why install auditd for Wazuh / SIEM?**
- ❖ auditd generates **detailed security logs**
- ❖ These logs are collected by:
  - ❖ **Wazuh Agent / Manager**
- ❖ Helps in:
  - Insider threat detection
  - File integrity monitoring
  - Compliance (PCI-DSS, HIPAA, ISO 27001)
- **What -y means**
- ❖ Automatically answers **Yes** to installation prompts
- ❖ Useful for **automation and scripts**

Wazuh Manager Main Configuration (ossec.conf)



- **Command Used for ossec.conf:**
  - ➢ sudo cat /var/ossec/etc/ossec.conf
- **Purpose**
- Displays the **main configuration file** of **Wazuh Manager**
- Controls how:
  - Logs are processed
    - ➢ Alerts are generated
    - ➢ Notifications are sent
    - ➢ Agents communicate with Manager
- **Global Configuration Section**
- **<global>**
  <jsonout_output>yes</jsonout_output>
- **<alerts_log>yes</alerts_log>**
- **<logall>no</logall>**
- **<email_notification>no</email_notification>**
- **</global>**

Explanation

- Enables **JSON output** for alerts (used by SIEM dashboards)
- Stores alerts in alerts.log
- Disables full log storage to save disk space
- Email alerts are disabled by default

Alert Notification Integration (Slack)

- **<integration>**

  <name>slack</name>

  <hook_url>https://hooks.slack.com/...</hook_url>

  <alert_format>json</alert_format>

  </integration>

Explanation

- Integrates Wazuh alerts with **Slack**
- Sends real-time security alerts to SOC team
- Uses JSON format for structured alert data

Alert Levels Configuration

- <alerts>

  <log_alert_level>3</log_alert_level>

  <email_alert_level>12</email_alert_level>

  </alerts>

Explanation

- Alerts with severity **level ≥ 3** are logged
- Only **critical alerts (≥12)** are eligible for email
- Helps reduce alert noise

Logging Format

- <logging>

  <log_format>plain</log_format>

  </logging>

Explanation

- Internal Wazuh logs are in **plain text**
- Easier for debugging in terminal
- JSON can be enabled if needed

Agent Communication Settings

- <remote>

  <connection>secure</connection>

  <port>1514</port>

  <protocol>tcp</protocol>

  </remote>

Explanation

- Secure communication between:
  - Wazuh Manager and Agents
- Uses:
  - TCP protocol
  - Port **1514**
  - Encrypted connection

Wazuh Security Monitoring Modules (ossec.conf)

```xml
<!-- Policy monitoring -->
<rootcheck>
  <disabled>no</disabled>
  <check_files>yes</check_files>
  <check_trojans>yes</check_trojans>
  <check_dev>yes</check_dev>
  <check_sys>yes</check_sys>
  <check_pids>yes</check_pids>
  <check_ports>yes</check_ports>
  <check_if>yes</check_if>

  <!-- Frequency that rootcheck is executed - every 12 hours -->
  <frequency>43200</frequency>

  <rootkit_files>etc/rootcheck/rootkit_files.txt</rootkit_files>
  <rootkit_trojans>etc/rootcheck/rootkit_trojans.txt</rootkit_trojans>

  <skip_nfs>yes</skip_nfs>
</rootcheck>

<wodle name="cis-cat">
  <disabled>yes</disabled>
  <timeout>1800</timeout>
  <interval>1d</interval>
  <scan-on-start>yes</scan-on-start>

  <java_path>wodles/java</java_path>
  <ciscat_path>wodles/ciscat</ciscat_path>
</wodle>

<!-- Osquery integration -->
<wodle name="osquery">
  <disabled>yes</disabled>
  <run_daemon>yes</run_daemon>
  <log_path>/var/log/osquery/osqueryd.results.log</log_path>
  <config_path>/etc/osquery/osquery.conf</config_path>
  <add_labels>yes</add_labels>
</wodle>

<!-- System inventory -->
<wodle name="syscollector">
  <disabled>no</disabled>
  <interval>1h</interval>
  <scan_on_start>yes</scan_on_start>
  <hardware>yes</hardware>
  <os>yes</os>
  <network>yes</network>
```

- **Rootcheck – Policy s Rootkit Monitoring**
  - <rootcheck>
- **Purpose**
  - Detects **rootkits, trojans, malware, and policy violations**
- **What is Enabled**
  - File integrity checks
  - Trojan C rootkit detection
  - System device checks
  - Running process (PIDs) checks
  - Open ports and network interface checks
- **Frequency**
  - Runs **every 12 hours** (43200 seconds)
- **Why Important**
  - Identifies stealthy attacks and privilege escalation
  - Detects unauthorized system modifications

CIS-CAT – Compliance Scanning

- <wodle name="cis-cat">

Purpose

- Collects **hardware, OS, and network inventory**

What It Gathers

- CPU, RAM, disks
- Installed OS C packages
- Network interfaces and configuration

Benefit

- Asset visibility in SOC
- Detects unauthorized system changes

Wazuh Inventory, Configuration Assessment s Vulnerability Settings

```xml
<!-- System inventory -->
<wodle name="syscollector">
  <disabled>no</disabled>
  <interval>1h</interval>
  <scan_on_start>yes</scan_on_start>
  <hardware>yes</hardware>
  <os>yes</os>
  <network>yes</network>
  <packages>yes</packages>
  <ports all="no">yes</ports>
  <processes>yes</processes>

  <!-- Database synchronization settings -->
  <synchronization>
    <max_eps>10</max_eps>
  </synchronization>
</wodle>

<sca>
  <enabled>yes</enabled>
  <scan_on_start>yes</scan_on_start>
  <interval>12h</interval>
  <skip_nfs>yes</skip_nfs>
</sca>

<vulnerability-detector>
  <enabled>no</enabled>
  <interval>5m</interval>
  <min_full_scan_interval>6h</min_full_scan_interval>
  <run_on_start>yes</run_on_start>

  <!-- Ubuntu OS vulnerabilities -->
  <provider name="canonical">
    <enabled>no</enabled>
    <os>trusty</os>
    <os>xenial</os>
    <os>bionic</os>
    <os>focal</os>
    <os>jammy</os>
    <update_interval>1h</update_interval>
  </provider>
```

- Syscollector – System Inventory
  - <wodle name="syscollector">
- **Purpose**
  - Collects **complete asset inventory** of the system
- **What is Collected**
  - Hardware details (CPU, RAM, disks)
  - Operating system information
  - Network interfaces
  - Installed packages
  - Running processes
  - Open ports
- **Behavior**
  - Runs **every 1 hour**
  - Scans immediately on system start
  - Helps SOC teams track system changes

Database Synchronization

- <synchronization>
  <max_eps>10</max_eps>
  </synchronization>

Purpose

- Controls how fast inventory data is sent to the database
- Prevents performance overload
- EPS = Events Per Second

SCA – Security Configuration Assessment

- <sca>

Purpose

- Checks system against **security hardening benchmarks**
- Identifies misconfigurations

Configuration

- Enabled
- Runs every **12 hours**
- Skips NFS-mounted files

Use Case

- Detects weak security settings
- Helps with compliance audits

Vulnerability Detector

- <vulnerability-detector>

Purpose

- Scans installed packages for **known CVEs**

Status

- Disabled by default
- Can be enabled for vulnerability scanning

Scan Behavior

- Quick scans every 5 minutes
- Full scan every 6 hours
- Runs automatically on startup

OS Vulnerability Providers (Ubuntu)

- <provider name="canonical">

Purpose

- Uses **Canonical security feeds**
- Detects vulnerabilities for Ubuntu OS versions:
  - Trusty, Xenial, Bionic, Focal, Jammy

Update Interval

Updates vulnerability data every **1 hour**

Wazuh OS-Specific Vulnerability Providers (Explanation)

This section tells Wazuh **which operating systems' vulnerability feeds it should use** to detect **CVE- based risks**.

Debian OS Vulnerabilities

- **<provider name="debian">**

What it does

Pulls vulnerability data from **Debian Security Advisories**

Checks installed packages against known CVEs

Supported Debian Versions

- Buster
- Bullseye
- Bookworm

```
<!-- Debian OS vulnerabilities -->
<provider name="debian">
  <enabled>no</enabled>
  <os>buster</os>
  <os>bullseye</os>
  <os>bookworm</os>
  <update_interval>1h</update_interval>
</provider>

<!-- RedHat OS vulnerabilities -->
<provider name="redhat">
  <enabled>no</enabled>
  <os>5</os>
  <os>6</os>
  <os>7</os>
  <os>8</os>
  <os>9</os>
  <update_interval>1h</update_interval>
</provider>

<!-- Amazon Linux OS vulnerabilities -->
<provider name="alas">
  <enabled>no</enabled>
  <os>amazon-linux</os>
  <os>amazon-linux-2</os>
  <os>amazon-linux-2022</os>
  <os>amazon-linux-2023</os>
  <update_interval>1h</update_interval>
</provider>

<!-- SUSE OS vulnerabilities -->
<provider name="suse">
  <enabled>no</enabled>
  <os>11-server</os>
  <os>11-desktop</os>
  <os>12-server</os>
  <os>12-desktop</os>
  <os>15-server</os>
  <os>15-desktop</os>
  <update_interval>1h</update_interval>
</provider>
```

Status

Disabled (enabled=no)

Can be enabled when Debian agents are monitored

RedHat OS Vulnerabilities

- <provider name="redhat">

What it does

- Uses **Red Hat Security Data**
- Detects vulnerabilities specific to RHEL
- **Supported Versions**
- RHEL 5, 6, 7, 8, 9

Status

- Disabled by default
- Enabled only when RedHat systems are present

Amazon Linux Vulnerabilities (AWS)

- <provider name="alas">

What it does

- Uses **Amazon Linux Advisory System (ALAS)**
- Designed for **EC2 and cloud environments**

Supported Versions

- Amazon Linux
- Amazon Linux 2
- Amazon Linux 2022 C 2023

SUSE OS Vulnerabilities

- <provider name="suse">+

What it does

- Fetches vulnerabilities from **SUSE Security Advisories**

Supported Versions

- SUSE 11, 12, 15 (Server C Desktop)

Status

- Disabled by default

Why These Providers Are Disabled by Default

- Wazuh enables **only required providers**

Reduces:

  - False positives
  - Resource usage

Improves performance in SOC environments

- Providers are enabled **based on agent OS**

Update Behavior

- <update_interval>1h</update_interval>
- Vulnerability databases are refreshed **every 1 hour**
- Ensures latest CVEs are detected quickly

Wazuh uses OS-specific vulnerability providers to accurately detect known CVEs on different Linux distributions while minimizing false positives and performance overhead.

Vulnerability Detection s File Integrity Monitoring (Wazuh)

```
  <!-- Alma Linux OS vulnerabilities -->
  <provider name="almalinux">
    <enabled>no</enabled>
    <os>8</os>
    <os>9</os>
    <update_interval>1h</update_interval>
  </provider>

  <!-- Windows OS vulnerabilities -->
  <provider name="msu">
    <enabled>yes</enabled>
    <update_interval>1h</update_interval>
  </provider>

  <!-- Aggregate vulnerabilities -->
  <provider name="nvd">
    <enabled>yes</enabled>
    <update_interval>1h</update_interval>
  </provider>

</vulnerability-detector>

<!-- File integrity monitoring -->
<syscheck>
  <disabled>no</disabled>

  <!-- Frequency that syscheck is executed default every 12 hours -->
  <frequency>43200</frequency>

  <scan_on_start>yes</scan_on_start>

  <!-- Generate alert when new file detected -->
  <alert_new_files>yes</alert_new_files>

  <!-- Don't ignore files that change more than 'frequency' times -->
  <auto_ignore frequency="10" timeframe="3600">no</auto_ignore>

  <!-- Directories to check  (perform all possible verifications) -->
  <directories>/etc,/usr/bin,/usr/sbin</directories>
  <directories>/bin,/sbin,/boot</directories>
```

- This part tells Wazuh **where to get vulnerability (CVE) information from** and **for which operating systems**.
- **AlmaLinux Vulnerabilities**
  - <provider name="almalinux">
  - Used for **AlmaLinux servers**
  - Supports **AlmaLinux 8 and G**
  - Currently **disabled**
  - Enabled only if AlmaLinux agents exist
  - **Why disabled?**
    To avoid unnecessary CVE checks on non-AlmaLinux systems.
- **Windows Vulnerabilities (MSU)**
  - <provider name="msu">
  - Uses **Microsoft Security Update (MSU) feed**
  - Detects:
    - Missing Windows patches
    - OS-level Windows vulnerabilities
- **Enabled**
  - Updates CVE data **every 1 hour**

Aggregate Vulnerabilities (NVD)

- <provider name="nvd">

Uses **NIST National Vulnerability Database Covers:**

  - OS vulnerabilities
  - Application vulnerabilities
  - Third-party software CVEs

Enabled

- Most important and trusted CVE source

Why NVD is critical

- It provides CVSS scores and real-world exploit context.

Update Interval

- <update_interval>1h</update_interval>
- CVE databases refresh **every hour**
- Ensures near-real-time vulnerability detection

Syscheck – File Integrity Monitoring (FIM)

This is **host-based intrusion detection**.

What Syscheck Does

- Monitors **critical system files and directories**

Detects:

- File modification
- New file creation
- File deletion

Uses:

- Hash comparison
- Permission C ownership checks

Scan Frequency

- <frequency>43200</frequency>
- Runs every 12 hours
- Also runs at system startup

Noise Control (Anti Alert Flooding)

- <auto_ignore_frequency>
- Prevents repeated alerts for frequently changing files
- Time window: **1 hour**
- Keeps alerts meaningful for SOC analysts

Critical Directories Monitored

- /etc
- /usr/bin
- /usr/sbin
- /bin
- /sbin
- /boot

Wazuh Syscheck Ignore Rules s Active Response Configuration

Files s Directories to Ignore (Noise Reduction)

- <ignore>/etc/mtab</ignore>
- <ignore>/etc/hosts.deny</ignore>
- …
- <ignore>/etc/svc/volatile</ignore>

What this means

- These files **change frequently**
- Monitoring them would generate **too many false alerts**
- Wazuh **intentionally ignores** them

```
  <!-- Files/directories to ignore -->
  <ignore>/etc/mtab</ignore>
  <ignore>/etc/hosts.deny</ignore>
  <ignore>/etc/mail/statistics</ignore>
  <ignore>/etc/random-seed</ignore>
  <ignore>/etc/random.seed</ignore>
  <ignore>/etc/adjtime</ignore>
  <ignore>/etc/httpd/logs</ignore>
  <ignore>/etc/utmpx</ignore>
  <ignore>/etc/wtmpx</ignore>
  <ignore>/etc/cups/certs</ignore>
  <ignore>/etc/dumpdates</ignore>
  <ignore>/etc/svc/volatile</ignore>

  <!-- File types to ignore -->
  <ignore type="sregex">.log$|.swp$</ignore>

  <!-- Check the file, but never compute the diff -->
  <nodiff>/etc/ssl/private.key</nodiff>

  <skip_nfs>yes</skip_nfs>
  <skip_dev>yes</skip_dev>
  <skip_proc>yes</skip_proc>
  <skip_sys>yes</skip_sys>

  <!-- Nice value for Syscheck process -->
  <process_priority>10</process_priority>

  <!-- Maximum output throughput -->
  <max_eps>50</max_eps>

  <!-- Database synchronization settings -->
  <synchronization>
    <enabled>yes</enabled>
    <interval>5m</interval>
    <max_eps>10</max_eps>
  </synchronization>
</syscheck>

<!-- Active response -->
<global>
  <white_list>127.0.0.1</white_list>
  <white_list>^localhost.localdomain$</white_list>
  <white_list>192.168.1.1</white_list>
```

Why these are ignored

File / Directory	Reason
➢ `/etc/mtab`	Changes with mounts
➢ `/etc/utmp`, `/etc/wtmp`	Login records update often
➢ `/etc/random-seed`	Changes on every boot
➢ `/etc/httpd/logs`	Log files change continuously
➢ `/etc/cups/certs`	Printer service updates

File Types to Ignore (Regex-Based)

- <ignore type="sregex">.log$|.swp$</ignore>

What this does

- Ignores:
  - .log files
  - .swp (editor swap) files
- Uses **simple regex matching**

Nodiff – Monitor Without Content Comparison

- <nodiff>/etc/ssl/private.key</nodiff>

What it means

- File is monitored for:
  - Existence
  - Permission changes
- But **content difference is never calculated**

Why important

- Prevents sensitive data exposure
- Improves performance
- Protects private keys

Active Response – Whitelist Configuration

- <white_list>127.0.0.1</white_list>
- <white_list>localhost.localdomain</white_list>
- <white_list>192.168.1.1</white_list>

What this does

- Prevents **active response actions** against trusted IPs
- Protects:
  - Localhost
  - Internal network systems
  - Admin machines

Wazuh Active Response Configuration (Explanation)

What is Active Response in Wazuh?

- **Active Response = Automatic reaction to security alerts**
- Instead of only **detecting** attacks, Wazuh can:
- Block attackers
- Disable compromised users
- Cut malicious network access
- Take immediate action without human delay

This turns Wazuh from **SIEM → SOAR (basic level)**.

Whitelist (Protection for Trusted Systems)

- <white_list>127.0.0.1</white_list>
- <white_list>localhost.localdomain</white_list>
- <white_list>192.168.1.1</white_list>

What this does

- Prevents **active response actions** against:
  - Localhost
  - Trusted internal IPs
  - Admin machines

Why this is critical

- Avoids **self-lockout**
- Prevents blocking SOC/admin systems by mistake

```
<!— Active response —>
<global>
  <white_list>127.0.0.1</white_list>
  <white_list>^localhost.localdomain$</white_list>
  <white_list>192.168.1.1</white_list>
</global>

<command>
  <name>disable-account</name>
  <executable>disable-account</executable>
  <timeout_allowed>yes</timeout_allowed>
</command>

<command>
  <name>restart-wazuh</name>
  <executable>restart-wazuh</executable>
</command>

<command>
  <name>firewall-drop</name>
  <executable>firewall-drop</executable>
  <timeout_allowed>yes</timeout_allowed>
</command>

<command>
  <name>host-deny</name>
  <executable>host-deny</executable>
  <timeout_allowed>yes</timeout_allowed>
</command>

<command>
  <name>route-null</name>
  <executable>route-null</executable>
  <timeout_allowed>yes</timeout_allowed>
</command>

<command>
  <name>win_route-null</name>
  <executable>route-null.exe</executable>
  <timeout_allowed>yes</timeout_allowed>
</command>

<command>
  <name>netsh</name>
  <executable>netsh.exe</executable>
  <timeout_allowed>yes</timeout_allowed>
</command>
```

Active Response Commands (Linux)

- These commands define **WHAT action Wazuh can take**.

Disable User Account

- <name>disable-account</name>
- Disables a user account temporarily
- Used when:
  - Account compromise is detected
  - Brute-force login attempts occur

⏱ timeout_allowed=yes → account can be re-enabled automatically

Restart Wazuh

- <name>restart-wazuh</name>
- Restarts Wazuh services automatically
- Used after:
  - Configuration change
  - Service crash detection

Firewall Drop (Block IP)

- <name>firewall-drop</name>
- Blocks attacker IP using firewall rules
- Very common SOC response to:
  - Brute-force attacks
  - Port scanning
  - DDoS attempts

⏱ Timeout allows **temporary blocking**

Host Deny

- <name>host-deny</name>
- Blocks IP using /etc/hosts.deny
- Older but still effective protection method

Route Null (Linux)

- <name>route-null</name>
- Sends malicious traffic to a **null route**
- Effectively drops all packets from attacker 4

Active Response Commands (Windows)

Windows Route Null

- <name>win_route-null</name>
- Windows version of route blocking
- Drops traffic from malicious IP

Netsh (Windows Firewall)

- <name>netsh</name>
- Uses Windows Firewall (netsh)
- Blocks attacker IP at OS level

Wazuh Log Analysis, Rules s Rule Testing (Deep Explanation)

```
<!--
<active-response>
  active-response options here
</active-response>
-->

<!-- Log analysis -->
<localfile>
  <log_format>command</log_format>
  <command>df -P</command>
  <frequency>360</frequency>
</localfile>

<localfile>
  <log_format>full_command</log_format>
  <command>netstat -tulpn | sed 's/\([[:alnum:]]\+\)\ \+[[:digit:]]\+\ \+[[:digit:]]\+\ \+\(.*\):\([[:digit:]]*\)\ \+\([0-9\.\:\*]\+\).\+\ \([[:digit:]]*\/[[:alnum:]\-]*\).*/\1 \2 = \3 = \4 \5/' | sort -k 4 -g | sed 's/ = \(.*\
:\1/' | sed 1,2d</command>
  <alias>netstat listening ports</alias>
  <frequency>360</frequency>
</localfile>

<localfile>
  <log_format>full_command</log_format>
  <command>last -n 20</command>
  <frequency>360</frequency>
</localfile>

<ruleset>
  <!-- Default ruleset -->
  <decoder_dir>ruleset/decoders</decoder_dir>
  <rule_dir>ruleset/rules</rule_dir>
  <rule_exclude>0215-policy_rules.xml</rule_exclude>
  <list>etc/lists/audit-keys</list>
  <list>etc/lists/amazon/aws-eventnames</list>
  <list>etc/lists/security-eventchannel</list>

  <!-- User-defined ruleset -->
  <decoder_dir>etc/decoders</decoder_dir>
  <rule_dir>etc/rules</rule_dir>
</ruleset>

<rule_test>
  <enabled>yes</enabled>
  <threads>1</threads>
  <max_sessions>64</max_sessions>
  <session_timeout>15m</session_timeout>
</rule_test>
```

- **Active Response Block (Placeholder)**
- <active-response>
- active-response options here
- </active-response>
- **Meaning**
- This is where **active response rules are linked**
- Defines:
  - Which alerts trigger actions
  - Timeout values
  - Target (agent/server)
- Actual actions are defined earlier; this block **connects detection → response**
- **Log Analysis – Command-Based Log Collection**
- Wazuh can collect **command output as logs**, not just files.
- **Disk Usage Monitoring**
- <command>df -P</command>
- <frequency>360</frequency>
- **What it does**
- Runs df -P every **6 minutes**
- Monitors disk space usage

Listening Ports Monitoring

- <command>netstat -tuln</command>
- <alias>netstat listening ports</alias>

What it does

- Captures open/listening ports
- Detects:
  - New services
  - Backdoors
  - Unauthorized daemons

User Login Monitoring

- <command>last -n 20</command>

What it does

- Tracks recent login history
- Detects:
  - Suspicious logins
  - Unauthorized access

Ruleset Configuration – Detection Engine

- This defines **where Wazuh gets its detection logic**.

Default Ruleset

- <decoder_dir>ruleset/decoders</decoder_dir>
- <rule_dir>ruleset/rules</rule_dir>

Purpose

- Uses official Wazuh decoders and rules
- Covers:
  - Authentication
  - Malware
  - System misuse
  - Network attacks

Rule Exclusions

- <rule_exclude>0215-policy_rules.xml</rule_exclude>
- Excludes specific rule files
- Used to:
  - Reduce false positives
  - Customize detection

Lists

- <list>etc/lists/audit-keys</list>
- <list>etc/lists/amazon/aws-eventnames</list>

Purpose

- Lookup tables for rules
- Improve accuracy and speed

User-Defined Rules

- <decoder_dir>etc/decoders</decoder_dir>
- <rule_dir>etc/rules</rule_dir>

Purpose

- Allows **custom SOC rules**
- Enables detection of:
  - Organization-specific attacks
  - Custom applications

Rule Testing – Validation Engine

- <rule_test>

What it does

- Tests rules before applying them
- Prevents:
  - Invalid rules
  - Broken configurations

Configuration

- Enabled
- Single-threaded
- Max 64 sessions
- 15-minute timeout

Rule Testing, Agent Authentication s Clustering (Wazuh)

```
  <rule_test>
    <enabled>yes</enabled>
    <threads>1</threads>
    <max_sessions>64</max_sessions>
    <session_timeout>15m</session_timeout>
  </rule_test>

  <!-- Configuration for wazuh-authd -->
  <auth>
    <disabled>no</disabled>
    <port>1515</port>
    <use_source_ip>no</use_source_ip>
    <purge>yes</purge>
    <use_password>no</use_password>
    <ciphers>HIGH:!ADH:!EXP:!MD5:!RC4:!3DES:!CAMELLIA:@STRENGTH</ciphers>
    <!-- <ssl_agent_ca></ssl_agent_ca> -->
    <ssl_verify_host>no</ssl_verify_host>
    <ssl_manager_cert>etc/sslmanager.cert</ssl_manager_cert>
    <ssl_manager_key>etc/sslmanager.key</ssl_manager_key>
    <ssl_auto_negotiate>no</ssl_auto_negotiate>
  </auth>

  <cluster>
    <name>wazuh</name>
    <node_name>node01</node_name>
    <node_type>master</node_type>
    <key></key>
    <port>1516</port>
    <bind_addr>0.0.0.0</bind_addr>
    <nodes>
        <node>NODE_IP</node>
    </nodes>
    <hidden>no</hidden>
    <disabled>yes</disabled>
  </cluster>

</ossec_config>

<ossec_config>
  <localfile>
    <log_format>apache</log_format>
    <location>/var/log/nginx/access.log</location>
  </localfile>

  <localfile>
    <log_format>apache</log_format>
```

- **Rule Testing Configuration (<rule_test>)**
- <rule_test>
- <enabled>yes</enabled>
- <threads>1</threads>
- <max_sessions>64</max_sessions>
- <session_timeout>15m</session_timeout>
- </rule_test>
- ◈ **What this does**
- Enables **rule validation engine**
- Checks rules **before they are applied**
- ◈ **Why important**
- Prevents:
  - Invalid XML rules
  - Misconfigured detection logic
  - Wazuh Manager crashes

Agent Authentication Service (wazuh-authd)

- <auth>

Purpose

- Handles **secure agent registration**
- Verifies agents before allowing them to send logs

Setting	Meaning
Key Settings Explained	
`disabled=no`	Authentication service enabled
`port=1515`	Agent enrollment port
`use_source_ip=no`	Agent IP not strictly enforced
`purge=yes`	Old agent entries cleaned
`use_password=no`	No password-based auth

Security (SSL/TLS)

- <ciphers>HIGH:!ADH:!EXP...</ciphers>
- <ssl_verify_host>no</ssl_verify_host>
- Strong encryption ciphers used
- SSL certificates ensure secure communication
- Prevents MITM attacks during agent enrolment

Wazuh Cluster Configuration

- <cluster>

What clustering does

- Allows **multiple Wazuh Managers**

Enables:

  - High availability
  - Load balancing
  - Scalability

Cluster Settings Explained

Parameter	Meaning
`name`	Cluster name
`node_name`	Current node ID
`node_type`	Master node
`port=1516`	Cluster communication port
`bind_addr=0.0.0.0`	Listen on all interfaces
`nodes`	Other manager nodes
`hidden=no`	Node visible
`disabled=yes`	Cluster currently OFF

Local Log Collection (Web Server Logs)

- <localfile>
- <log_format>apache</log_format>
- <location>/var/log/nginx/access.log</location>
- </localfile>

What this does

- Collects **NGINX/Apache access logs**
- Enables detection of:
  - Web attacks
  - SQL injection
  - Brute-force attempts

Wazuh Local Log Collection (Web s System Logs)

This section tells Wazuh **which log files to read and analyze**.

NGINX Web Server Logs

- <log_format>apache</log_format>
- <location>/var/log/nginx/access.log</location>

What it does

- Collects **NGINX access logs**
- Uses **Apache log format** (NGINX compatible)

What Wazuh detects

- SQL Injection attempts
- Brute-force login attempts
- Suspicious URLs
- Web scanning C bots

```
che</log_format>
og/nginx/access.log</location>

che</log_format>
og/nginx/error.log</location>

che</log_format>
og/apache2/error.log</location>

che</log_format>
og/apache2/access.log</location>

og</log_format>
ssec/logs/active-responses.log</location>

og</log_format>
og/dpkg.log</location>
```

- <location>/var/log/nginx/error.log</location>

Purpose

- Monitors NGINX error logs
- Detects:
  - Application crashes
  - Misconfigurations
  - Exploitation errors

Apache Web Server Logs

- <location>/var/log/apache2/access.log</location>

What it does

- Monitors Apache access logs

Detects web attacks in real time

- <location>/var/log/apache2/error.log</location>

What it does

- Monitors Apache error logs
- Helps detect:
  - Failed exploits
  - Server-side issues
  - Misuse of web services

Active Response Logs

- <log_format>syslog</log_format>
- <location>/var/ossec/logs/active-responses.log</location>

Purpose

- Logs **all automated actions** taken by Wazuh
- Example:
  - IP blocked
  - User disabled
  - Firewall rule applied

Package Management Logs (dpkg)

- <location>/var/log/dpkg.log</location>

What it does

- Tracks:
  - Software installation
  - Package removal
  - System updates

Wazuh Active Response – Windows Firewall Blocking

What this Configuration Is About

- This configuration enables **automatic IP blocking on Windows systems** using **Windows Firewall** when a security alert is triggered.
- It is part of **Active Response (SOAR behavior)** in Wazuh.

Command Definition (What Action to Run)

- <command>

  <name>windows_firewall</name>
  <executable>windows_firewall.cmd</executable>

  <timeout_allowed>no</timeout_allowed>

  </command>

Explanation

- **name**: Logical name of the response action
- **executable**: Script executed on the Windows agent → windows_firewall.cmd blocks attacker IP
- **timeout_allowed=no**:
  - Block is **permanent**
  - Manual removal required

Active Response Configuration (When s Where to Run)

- <active-response>
- <disabled>no</disabled>
- <command>windows_firewall</command>
- <location>local</location>
- <timeout>0</timeout>
- </active-response>

Explanation

- **disabled=no** → Active response enabled
- **command** → Calls windows_firewall command
- **location=local** → Runs on the **Windows agent**
- **timeout=0** → No auto-unblock (permanent block)

SOC Workflow (Easy to Explain)

- Attack detected (e.g. brute-force login)
- Wazuh rule is triggered
- Active response executes
- Windows Firewall blocks attacker IP automatically
- SOC reviews the alert in dashboard

Enabling Active Response in Wazuh (Step-by-Step)

Editing Main Configuration (ossec.conf)

- nano /var/ossec/etc/ossec.conf

What this step does

- Opens the **main Wazuh Manager configuration file**
- Used to:
  - Enable Active Response
  - Define commands and response behavior

This is the **core control file** of Wazuh Manager

Restarting Wazuh Manager

- systemctl restart wazuh-manager

Why this is required

- Applies all configuration changes
- Reloads:
  - Rules
  - Active response settings
  - Command definitions

Editing Active Response Rules File

- nano /var/ossec/etc/rules/0900-active_response.xml

What this file is for

- Contains **rules that trigger active responses**
- Defines:
  - Which alert ID or condition
  - Triggers which response (firewall block, user disable, etc.)

Configuring s Activating Active Response in Wazuh

Editing Wazuh Main Configuration

- nano /var/ossec/etc/ossec.conf

Explanation

- Opens the **main Wazuh Manager configuration file**
- Used to:
  - Enable Active Response
  - Define response commands
  - Configure response behavior

This file controls the **core behavior of Wazuh**

Restarting Wazuh Manager

- systemctl restart wazuh-manager

Explanation

- Opens the **main Wazuh Manager configuration file**
- Used to:
  - Enable Active Response
  - Define response commands
  - Configure response behavior

This file controls the **core behavior of Wazuh**

Editing Active Response Rules File

- nano /var/ossec/etc/rules/600000-active_response.xm

Explanation

- Opens the **Active Response rules file**
- This file defines:
  - Which alerts trigger responses
  - What action to take (IP block, firewall drop, etc.)
- Acts as the **link between detection and response**

Wazuh Active Response Rule – Windows Firewall

This rule is written inside

- /var/ossec/etc/rules/600000-active_response.xml

Line-by-Line Explanation

Rule Group

- <group name="active_response,">
- Groups this rule under **active_response**
- Helps Wazuh organize and categorize alerts

Used by SOC teams for filtering and reporting **Rule**

Definition

- <rule id="600000" level="10">
- **id="600000"** → Custom rule ID (safe range for user rules)
- **level="10"** → High-severity alert (Serious security incident)

Log Format

- <decoded_as>json</decoded_as>
- Tells Wazuh the log/event is in **JSON format**
- Required for structured Active Response events

Trigger Condition

- <field name="active_response">windows_firewall</field>
- This is the **key trigger**
- When the field active_response contains windows_firewall
- → This rule gets matched

This connects the **rule** to the **Windows firewall command**

Alert Description

- <description>Windows Firewall Active Response triggered.</description>
- Human-readable alert message
- Shown in:
  - Wazuh Dashboard
  - Alerts logs

Helps SOC analysts understand what happened instantly

SOC Group Tag

- <group>socfortress,</group>
- Custom SOC tag
- Used for:
  - Alert categorization
  - SOC dashboards
  - Incident reports

Logging Optimization

- <options>no_full_log</options>
- Prevents storing full raw logs
- Saves disk space
- Keeps alert clean and readable

Rule Closure

- </rule>
- </group>
- Ends the rule and group definition

Wazuh Active Response – Windows Firewall (End-to-End)

Command Definition (What action Wazuh can run)

```
<command>
  <name>windows_firewall</name>

 <executable>windows_firewall.exe</executable
 >
  <timeout_allowed>no</timeout_allowed>
</command>
```

Explanation

- Defines a **custom active response command**
- windows_firewall.exe runs on the **Windows agent**
- Blocks the attacker IP using **Windows Firewall**
- timeout_allowed=no → block is **permanent** (manual removal)

Active Response Rule (Audit s alerting)

File: /var/ossec/etc/rules/600000-active_response.xml

```
<group name="active_response,">
  <rule id="600000" level="10">
    <decoded_as>json</decoded_as>
    <field name="active_response">windows_firewall</field>
    <description>Windows Firewall Active Response triggered.</description>
    <group>socfortress,</group>
    <options>no_full_log</options>
  </rule>
</group>
```

Explanation

- Custom rule with **high severity (level 10)**
- Triggers when windows_firewall active response executes
- Generates a **SOC alert** confirming the response action
- Provides **audit trail** for automated blocking

Wazuh Rules Directory s File Ownership

Wazuh Rules Directory s File Ownership

- **Directory:** /var/ossec/etc/rules/
- This folder contains **all Wazuh detection and response rules**.

What this list shows

- You can see multiple rule files like:
- 500000-active_response.xml
- 600000-active_response.xml
- local_rules.xml
- exclusion_rules.xml
- win_sigma_rules_builtin.xml
- Each file contains **specific detection logic** (alerts, responses, exclusions).

Important Observation (VERY KEY POINT)

- Notice the ownership difference:
- -rw-r--r-- 1 root root 317 600000-active_response.xml
- -rw-rw---- 1 wazuh wazuh 316 500000-active_response.xml

⚠ **Problem**

- 600000-active_response.xml is owned by **root**
- Wazuh Manager runs as **wazuh user**
- Result:
  ☞ Wazuh **may not load or apply** this rule

Fixing Ownership (Command Used)

- chown wazuh:wazuh 600000-active_response.xml

What this does

- Changes file owner to:
  - **User:** wazuh
  - **Group:** wazuh
- Ensures Wazuh Manager can:
  - Read the rule
  - Apply the rule
  - Trigger active response correctly

Why This Step Is Critical (Exam + SOC Point)

- If rule ownership is wrong:
  - Rule is ignored
  - Active response won’t trigger
  - No alert appears in dashboard

Editing Main Configuration

- nano /var/ossec/etc/ossec.conf

Explanation

- Opens the main Wazuh Manager configuration file
- Used to enable/configure **Active Response**
- Any change here controls how Wazuh behaves globally

Restarting Wazuh Manager

- systemctl restart wazuh-manager

Explanation

- Applies all configuration changes
- Reloads:
  - Rules
  - Active response settings
- Mandatory step after editing configs

Editing Custom Active Response Rule

- nano /var/ossec/etc/rules/600000-active_response.xml

Explanation

- Opens the custom rule file
- This rule defines **when an active response alert is generated**
- Links detection → automated response → alert visibility

Navigating to Rules Directory

- cd rules/

Explanation

- Moves into the Wazuh rules directory
- Contains:
  - Default rules
  - Custom SOC rules
  - Active response rules

Wazuh Alert Notification in Slack (Auditd Event)

What is happening here

- Wazuh detected a **Linux audit event**
- The alert was **sent automatically to Slack**
- This confirms **real-time alerting s integration is working**

Alert Details Explained

- **Rule ID:** 80785 (Level 3)
- Rule triggered from **Auditd**
- Severity **Level 3** → Informational / low-risk activity
- **Alert Type:**
  Audit: Watch - Read access

Meaning:

- A file or directory being **monitored by auditd** was **read/accessed**
- This helps track:
  - File access
  - User behavior
  - Insider activity

Technical Information (From Alert) User: kali

Command executed: /usr/bin/zsh

System call: openat

Working directory: /home/kali

Log source: /var/log/audit/audit.log

Agent: (002) - akku

Editing Wazuh Agent Configuration

vi /var/ossec/etc/ossec.conf

Explanation

- Opens the Wazuh Agent configuration file
- Used to enable:
  - Audit log collection
  - Log forwarding to Wazuh Manager
- This ensures the agent knows what logs to monitor and send

Restarting Wazuh Agent

- sudo systemctl restart wazuh-agent

Explanation

- Applies changes made in ossec.conf
- Restarts the agent so:
  - New configurations are loaded
  - Log collection resumes correctly

```
┌──(root㉿kali)-[/home]
└─# vi /var/ossec/etc/ossec.conf

┌──(root㉿kali)-[/home]
└─# sudo systemctl restart wazuh-agent


┌──(root㉿kali)-[/home]
└─# exit

┌──(kali㉿kali)-[/etc/audit]
└─$ sudo cat /etc/audit/audit.rules

[sudo] password for kali:
## This file is automatically generated from /etc/audit/rules.d
-D
-b 8192
-f 1
--backlog_wait_time 60000
-w /home -p w -k audit-wazuh-w
-w /home -p a -k audit-wazuh-a
-w /home -p r -k audit-wazuh-r
-w /home -p x -k audit-wazuh-x


┌──(kali㉿kali)-[/etc/audit]
└─$
```

Viewing Auditd Rules

- sudo cat /etc/audit/audit.rules
- **Explanation**
- Displays **Linux audit rules** used by auditd
- These rules define **what system activity is monitored**

Viewing Auditd Rules

- sudo cat /etc/audit/audit.rules

Explanation

- Displays **Linux audit rules** used by auditd
- These rules define **what system activity is monitored**

Testing Auditd Rules (Verification)

```
┌──(root㉿kali)-[/home]
└─# touch /home/testfile

┌──(root㉿kali)-[/home]
└─# ls
kali  land  testfile

┌──(root㉿kali)-[/home]
└─# auditctl -l

-w /home -p w -k audit-wazuh-w
-w /home -p a -k audit-wazuh-a
-w /home -p r -k audit-wazuh-r
-w /home -p x -k audit-wazuh-x

┌──(root㉿kali)-[/home]
└─# ausearch -k audit-wazuh-w

────
time→Thu Jan 22 08:59:58 2026
type=PROCTITLE msg=audit(1769090398.545:479): proctitle=2F7362696E2F617564697
463746C002D52002F6574632F61756469742F61756469742E72756C6573
type=SYSCALL msg=audit(1769090398.545:479): arch=c000003e syscall=44 success=
yes exit=1076 a0=3 a1=7ffcefd350e0 a2=434 a3=0 items=0 ppid=29176 pid=29203 a
uid=1000 uid=0 gid=0 euid=0 suid=0 fsuid=0 egid=0 sgid=0 fsgid=0 tty=pts2 ses
=2 comm="auditctl" exe="/usr/sbin/auditctl" subj=unconfined key=(null)
type=CONFIG_CHANGE msg=audit(1769090398.545:479): auid=1000 ses=2 subj=unconf
ined op=add_rule key="audit-wazuh-w" list=4 res=1
────
time→Thu Jan 22 09:00:09 2026
```

- **Creating a Test File**
  - touch /home/testfile
- **Explanation**
  - Creates a new file in /home
  - This action is intentional to **trigger auditd rules**
  - Tests whether **write (w) monitoring** is working
- **Verifying File Creation**
  - ls
- **Explanation**
  - Confirms that testfile was successfully created
  - Ensures the test action actually occurred
- **Checking Active Audit Rules**
  - auditctl -l
- **Explanation**
  - Lists **currently active auditd rules**
  - Confirms that /home is being monitored for:
    - w → write
    - a → attribute change
    - r → read
    - x → execute

Searching Audit Logs (Key Step)

- ausearch -k audit-wazuh-w

Explanation

- Searches audit logs using the **key name**
- audit-wazuh-w → write activity in /home
- Filters only **relevant audit evens**

Understanding the Audit Output

From the output:

- **syscall=44** → file operation
- **success=yes** → operation succeeded
- **exe=/usr/bin/auditctl** → command executed
- **uid=1000 (kali)** → user who performed the action
- **file created in /home**

Auditd Write Event Testing (Second Verification)

```
┌──(root㉿kali)-[/home]
└─# cd

┌──(root㉿kali)-[~]
└─# touch /home/sachinnn


┌──(root㉿kali)-[~]
└─# ausearch -k audit-wazuh-w

----
time->Thu Jan 22 08:59:58 2026
type=PROCTITLE msg=audit(1769090398.545:479): proctitle=2F7362696E2F617564697
463746C002D52002F6574632F61756469742F61756469742E72756C6573
type=SYSCALL msg=audit(1769090398.545:479): arch=c000003e syscall=44 success=
yes exit=1076 a0=3 a1=7ffcefd350e0 a2=434 a3=0 items=0 ppid=29176 pid=29203 a
uid=1000 uid=0 gid=0 euid=0 suid=0 fsuid=0 egid=0 sgid=0 fsgid=0 tty=pts2 ses
=2 comm="auditctl" exe="/usr/sbin/auditctl" subj=unconfined key=(null)
type=CONFIG_CHANGE msg=audit(1769090398.545:479): auid=1000 ses=2 subj=unconf
ined op=add_rule key="audit-wazuh-w" list=4 res=1
----
time->Thu Jan 22 09:00:09 2026
type=PROCTITLE msg=audit(1769090409.816:488): proctitle=2F7362696E2F617564697
463746C002D52002F6574632F61756469742F61756469742E72756C6573
type=SOCKADDR msg=audit(1769090409.816:488): saddr=100000000000000000000000
type=SYSCALL msg=audit(1769090409.816:488): arch=c000003e syscall=44 success=
yes exit=1076 a0=3 a1=7ffd9850da40 a2=434 a3=0 items=0 ppid=29311 pid=29328 a
uid=4294967295 uid=0 gid=0 euid=0 suid=0 fsuid=0 egid=0 sgid=0 fsgid=0 tty=(n
```

- **Creating Another Test File**
  - touch /home/sachinnn
- **Explanation**
  - Creates a new file in /home
  - This action is done to **trigger the auditd write (w) rule**
  - Confirms monitoring works for **any file**, not just one test case
- **Searching Audit Logs by Key**
  - ausearch -k audit-wazuh-w
- **Explanation**
  - Searches audit logs using the **key audit-wazuh-w**
  - Filters only **write-related events** in /home
  - Used to validate that auditd captured the activity

Understanding the Output (Important)

From the log output:

- **type=SYSCALL** → system call related to file operation
- **success=yes** → file creation was successful
- **uid=1000 (kali)** → user who created the file
- **exe=/usr/bin/auditctl** → command involved
- **time stamp shown** → exact time of activity

Why This Test Is Important

- Confirms auditd rules are:
  - Active
  - Persistent
  - Correctly configured
- Ensures Wazuh will:
  - Receive these logs
  - Analyze them
  - Generate alerts if rules match